



Trabajo Autónomo 2

Proceso de Desarrollo

Repositorio de Código, Diagramas de Flujo y Codificación en Python

Estudiante: Angel Xavier Rezabala Perez

Institución: Universidad Internacional del Ecuador

Ingenierías Digitales y Tecnologías Emergentes

Carrera: Ingeniería en Inteligencia Artificial

Asignatura: Lógica de Programación

Docente: Heredia Jiménez Estefania Vanessa

Introducción

El presente documento describe el proceso completo de desarrollo del proyecto "Adivina el Número", correspondiente al Trabajo Autónomo 2 de la materia Lógica de Programación. El sistema implementa un juego en el que la computadora adivina, mediante el algoritmo de búsqueda binaria, el número que el jugador está pensando entre 1 y 100.

Se documenta aquí cada etapa del trabajo: la creación y configuración del repositorio en GitHub, la instalación de Git y su enlace con Visual Studio Code, los diagramas de flujo que sirvieron de base para la codificación, el software y las herramientas utilizadas, el tipo de variables y estructuras de control empleadas en el código, y finalmente las conclusiones obtenidas a partir del desarrollo del proyecto.

Creación del repositorio en GitHub

El primer paso del desarrollo consistió en crear un repositorio público en GitHub llamado LP-AUTONOMO-2, bajo la cuenta del estudiante (Angel-Rezabala). Este repositorio funciona como el espacio central de almacenamiento del proyecto, incluyendo el código fuente, los diagramas de flujo y la documentación (README).

Estructura del repositorio

- /diagramas - Diagramas de flujo del sistema (PDF e imágenes)
- /src - Código fuente en Python (AdivinaNumero.py)
- .gitignore - Archivos y carpetas excluidos del control de versiones
- README.md - Documentación general del proyecto

La carga inicial de los diagramas y del código se realizó de forma manual a través del navegador, utilizando la opción "Agregar archivo → Cargar archivos" que ofrece la interfaz web de GitHub.

Descarga de Git y enlace con Visual Studio Code

Para poder trabajar de forma directa desde el editor de código y mantener el repositorio sincronizado, fue necesario instalar Git en el equipo y conectarlo con Visual Studio Code.

Instalación de Git

Se descargó el instalador oficial desde git-scm.com/downloads y se completó la instalación con la configuración predeterminada. Se verificó la instalación correcta ejecutando el siguiente comando en la terminal:

```
git --version  
# git version 2.55.0.windows.3
```

Configuración de identidad

Antes de poder registrar cambios (commits), se configuró el nombre de usuario y el correo asociados a Git, de forma global en el equipo:

```
git config --global user.name "Angel Rezabala"  
git config --global user.email "anrezabalape@uide.edu.ec"
```

Clonado del repositorio

Desde Visual Studio Code, mediante la paleta de comandos (Git: Clone), se clonó el repositorio existente utilizando la URL HTTPS del repositorio, lo que descargó una copia local sincronizada con GitHub:

```
https://github.com/Angel-Rezabala/LP-AUTONOMO-2.git
```

Tras el clonado, VS Code solicitó autenticación con la cuenta de GitHub (mediante navegador), quedando así el editor conectado directamente al repositorio. Esta conexión se evidencia en el panel de Source Control, donde se visualiza el nombre del repositorio y la rama principal (main).

Flujo de trabajo (commit y sincronización)

Una vez enlazado el repositorio, cualquier modificación realizada en el código desde VS Code se sube directamente a GitHub siguiendo estos pasos:

- Editar el archivo y guardar los cambios (Ctrl + S)
- Ir al panel "Source Control" y revisar los cambios detectados

- Escribir un mensaje descriptivo del cambio realizado
- Confirmar el cambio con el botón "Commit"
- Sincronizar con el repositorio remoto mediante "Sync Changes"

Esta prueba se realizó exitosamente, confirmando que los cambios locales se reflejan de inmediato en el repositorio remoto de GitHub.

Diagramas de flujo

Antes de iniciar la codificación se diseñaron dos diagramas de flujo que representan la lógica completa del sistema, siguiendo la simbología estándar (óvalos para inicio/fin, paralelogramos para entrada/salida, rectángulos para procesos y rombos para decisiones).

Diagrama #1 búsqueda binaria

Representa el algoritmo principal del juego: la computadora calcula un número medio entre un rango mínimo y máximo, y ajusta ese rango según la pista que da el jugador (mayor, menor o correcto), hasta acertar.

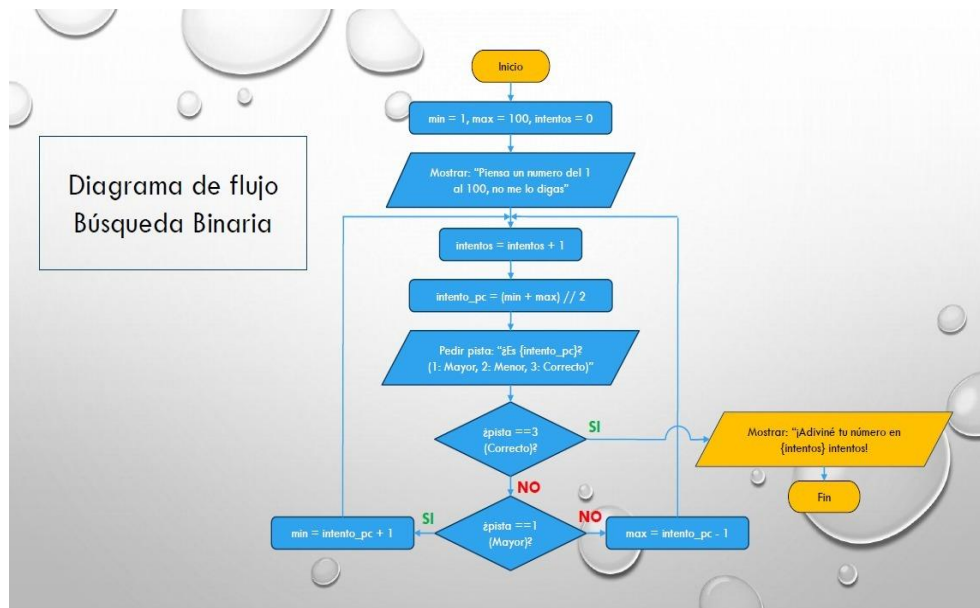


Figura 1. Diagrama de flujo Búsqueda Binaria

Diagrama #2 gestión de sesión

Representa el control del ciclo de partidas: al finalizar una partida, se pregunta al jugador si desea jugar de nuevo; según su respuesta, el sistema reinicia el juego o finaliza la ejecución mostrando un mensaje de despedida.

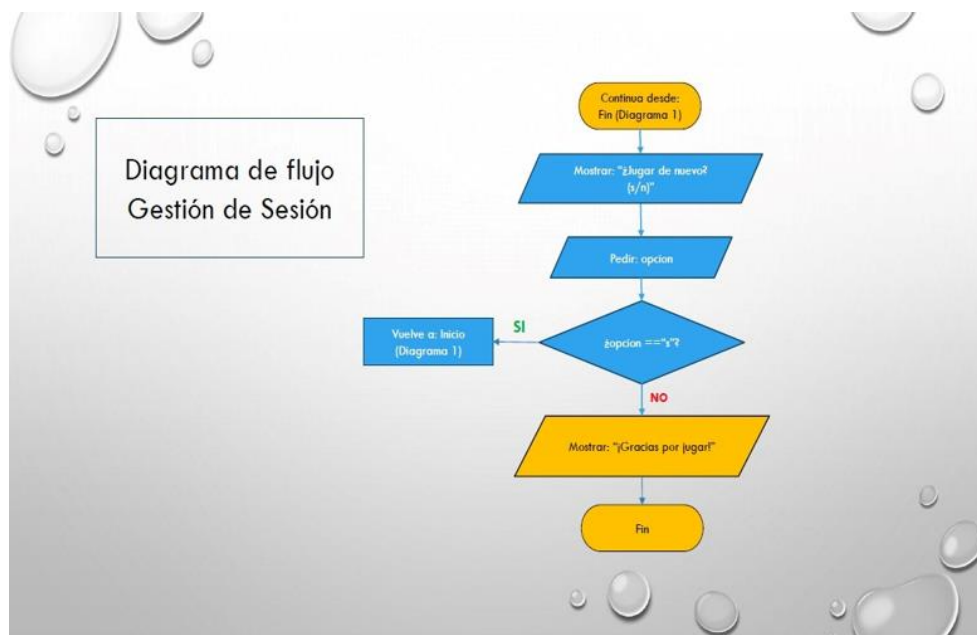


Figura 2. Diagrama de flujo Gestión de Sesión

Codificación del sistema

Software y herramientas utilizadas

- Lenguaje de programación: Python 3.13
- Entorno de desarrollo (IDE): Visual Studio Code
- Control de versiones: Git 2.55
- Repositorio remoto: GitHub
- Extensiones utilizadas: Python (Pylance) y visor de PDF

Tipo de variables utilizadas

Todas las variables del programa son de tipo entero (int), acordes a la naturaleza numérica del juego. Su nomenclatura respeta las reglas de Python (inician con letra, sin espacios, uso de guion bajo para separar palabras) y coincide exactamente con los nombres utilizados en los diagramas de flujo:

- min - límite inferior del rango de búsqueda
- max - límite superior del rango de búsqueda
- intentos - contador de intentos realizados
- intento_pc - número calculado por la computadora en cada intento
- pista - respuesta ingresada por el jugador (1, 2 o 3)
- opcion - respuesta del jugador para reiniciar o finalizar el juego

Estructuras de control aplicadas

El código aplica estructuras condicionales (if / elif / else) para evaluar la pista ingresada por el jugador y decidir el siguiente paso de la búsqueda binaria, y una estructura repetitiva (while) para mantener el ciclo de intentos y el ciclo de sesión de juego. Se utiliza además la instrucción break como alterador de bucle, para finalizar la repetición en el momento en que la computadora acierta el número.

```
print("Piensa un número del 1 al 100, no me lo digas.")
# Bucle principal (búsqueda binaria)
while True:
    intentos = intentos + 1
    intento_pc = (min + max) // 2

    print(f'\n¿Es {intento_pc}?')
    print("1: Mayor, 2: Menor, 3: Correcto")
    pista = int(input("Ingresa tu pista: "))

    # Rombo 1: ¿pista == 3 (Correcto)?
    if pista == 3:
        print(f"\n¡Adiviné tu número en {intentos} intentos!")
        break # Fin del juego

    # Rombo 2: ¿pista == 1 (Mayor)?
    elif pista == 1:
        min = intento_pc + 1
    else:
        max = intento_pc - 1
```

Fragmento representativo del código

Conclusión

El desarrollo de este proyecto permitió aplicar de forma integral los conocimientos adquiridos durante la materia de Lógica de Programación, abarcando desde el diseño de algoritmos mediante diagramas de flujo hasta su implementación en un lenguaje de programación.

Se comprobó que el código coincide exactamente con la lógica planteada en los diagramas. Asimismo, se logró establecer un flujo de trabajo mediante el uso de Git y GitHub, incluyendo tanto la carga manual de archivos a través del navegador como la sincronización directa desde Visual Studio Code, replicando prácticas utilizadas comúnmente en el desarrollo de software a nivel empresarial.

Finalmente, la aplicación del algoritmo de búsqueda binaria, en lugar de una búsqueda simple permitió optimizar el número de intentos necesarios para que el sistema adivine el número pensado por el jugador, demostrando la importancia de seleccionar algoritmos eficientes en el desarrollo de software.